In the context of property files, profiles are used to group configuration data, and they can be mapped to deployment environments without facing the same issues we analyzed in the previous section when using profiles as feature flags. But that applies only as long as you don’t package the profile-specific property files with the application. The 15-Factor methodology recommends not batching configuration values into groups named after environments and bundled with the application source code because it wouldn’t scale. As a project grows, new environments might be created for different stages; developers might create their own custom environments to try out new functionality. You can quickly end up with way too many configuration groups, implemented like Spring profiles and requiring new builds. Instead, you’ll want to keep them outside the application, such as in a dedicated repository served by a configuration server, as you’ll see later in this chapter. The only exception is for defaults and development-oriented configuration.

The following section will cover how Spring Boot addresses externalized configuration. You’ll learn how to use command-line arguments, JVM system properties, and environment variables to provide configuration data from the outside while using the same application build.

4.2 Externalized configuration: One build, multiple configurations

Property files bundled with the application source code are useful for defining some sensible defaults. Still, if you need to provide different values depending on the environment, you’ll need something else. Externalized configuration allows you to configure your application depending on where it’s deployed while consistently using the same immutable build for your application code. The critical aspect is that you don’t change your